たこ診 技術構成（フレームワーク）

> 版：1.0／作成日：2026-11-08／対応する仕様書：1.55
>
> この文書は、たこ診を動かしている技術の全体像を 1 冊にまとめたもの。細かい決まりは仕様書（`docs/SPEC.md`）の該当する節を正とし、ここでは「何を、なぜ、どう組み合わせているか」を説明する。食いちがったら仕様書とコードを正とする。

1. 全体像

たこ診は、ホストが作った診断に友達が匿名で答え、人数がそろった瞬間にサーバーが 1 回だけ結果を計算して保存する Web アプリである。画面と API を 1 つの Next.js アプリで作り、OpenNext で Cloudflare Workers 向けに変換して世界中の拠点で動かす。データは外部の PostgreSQL（Neon）に置く。

```
利用者（スマホのブラウザ・LINE などのアプリ内ブラウザ）
  │ https://takoshin.daigakunote.com（workers.dev のアドレスも可）
  ▼
Cloudflare Workers ― worker.ts（入口：https への移し替え・よそのサイトからの書き込みの拒否・Cron）
  └── OpenNext が変換した Next.js アプリ（App Router）
      ├── 画面（React。公開の前に作れるページは静的ファイルから返す）
      ├── API（/api/…。読み書きはすべてここを通す）
      └── 画像（next/og：結果のシェア画像・OG 画像）
  │ Hyperdrive（接続の仲立ち。キャッシュは切る）
  ▼
PostgreSQL（Neon。表は最初の接続のときに自動で作る）

周りの仕組み
  ├── Cron Trigger（1 日 1 回）→ /api/cron/cleanup：期限切れ・削除・計測データの掃除
  ├── Web Push の中継（Google・Mozilla・Apple・Microsoft）：「そろった」の通知
  ├── Stripe（任意）：特典つきの応援の支払いと Webhook
  └── GitHub Actions：CI・Deploy・Healthcheck（3 時間ごと）
```

設計の芯は次の 3 つで、以降の章はすべてこれを守るための組み立てである。

1. **結果は N 人目の回答が保存された瞬間に 1 回だけ計算し、以後は変えない。**それまで、途中の傾向を含めて結果に関わる情報をサーバーの外に出さない（仕様書 §1・§5.3）。
2. **だれが何と答えたかは、どの画面にも出さない。**1 人分の答えを推測できる値（軸の合計点・細かい時刻・3 人未満の内訳）は送らない（§8.5）。
3. **登録なし・個人情報なし。**鍵は URL の `#` より後ろ（サーバーに送られない）か、リクエストの見出しで渡し、記録に残さない（§8.1・§8.2）。

2. 使っている部品

部分	部品	版（2026-11-08）	選んだ理由
画面とサーバー処理	Next.js（App Router）＋ React ＋ TypeScript	Next 16.4.3、React 19.3.1、TypeScript 6.0.3	1 つの仕組みで画面と API と画像が作れる
置き場所	Cloudflare Workers（@opennextjs/cloudflare で変換）	OpenNext 1.20.14、wrangler 4.156	世界中の拠点で動き、無料の範囲が広い。Cron Trigger・Smart Placement・静的ファイルの配信がある
データベース	PostgreSQL（Neon）＋ `pg`。Hyperdrive 経由	pg 8.23.1	「N 人目まで」を行の鍵（`select … for update`）で確実に守れる。Workers からは問い合わせごとに接続するため仲立ちが要る
開発・テスト用 DB	PGlite（組み込みの PostgreSQL、WebAssembly）	PGlite 0.5.11	本物の PostgreSQL がなくても同じ SQL が動く。CI は PGlite と PostgreSQL 16 の両方で試す
結果画像	`next/og`（Satori）	Next に同梱	サーバーで PNG を作れる
通知	Web Push（RFC 8030・8291・8292）を Web Crypto だけで	外部の部品なし	Workers で動く部品が少なく、標準だけで組める
運営者の認証	パスキー（WebAuthn）＋ 合言葉	@simplewebauthn/server 14.0.3	指紋・顔認証で入れ、サーバーには公開鍵しか置かない
支払い（任意）	Stripe Checkout ＋ Webhook	REST を直接呼ぶ	特典つきの応援。使わなければ欄ごと出ない
QR コード	qrcode	qrcode 1.5.4	招待リンクの QR を端末の中で作る
品質の確認	vitest・eslint（eslint-config-next）・prettier・tsc	vitest 5.0.5、eslint 10.13.0、prettier 3.9.10	CI で全部を回す

依存部品は Dependabot が週 1 回提案する。**公開から 14 日たった版だけ**を取り込む（乗っ取られた版がまだ取り除かれていないおそれがあるため）。TypeScript 7 は提案させない設定にしている（§11.9）。

3. コードの構成

```
src/
  app/            画面と API（Next.js の App Router）
    page.tsx        S1 トップ          new/        S2 診断を作る
    h/              ホストの画面（share・自己回答 self・回答状況・result・more＝人数をふやす・thanks）
    a/[surveyId]/   回答者の画面（R1〜R3 を 1 つの URL の中で切り替える）
    r/[shareId]/    S7 公開カード・S8 2 人を並べる      types/   T1 16 タイプの紹介
    g/              G1・G2 グループ診断（新規作成は終了。既存は期限まで）
    admin/          運営者のページ       contact/ terms/ privacy/ legal/ uranai/
    api/            API（§6 を参照）
  components/     画面の部品（AnswerFlow＝1 画面 1 問、TypeCard、Meters、InviteShare、NotifyCard …）
  data/           データファイル：questions.ts（32 問）、types.json（16 タイプ）、framework.ts（軸・界隈・相性・関係）、
                  deepTypes.ts・deepAxes.ts（くわしい分析）、fortunes.ts（占い）、ng-words.ts、funnel.ts、seller.ts
  lib/            サーバーと共通の処理（下の表）
public/           sw.js（通知の受け取りだけ）、アイコン、タコの絵
```

```
worker.ts          Workers の入口（入口の決まり＋ Cron）
wrangler.jsonc     Workers の設定（独自ドメイン・Cron・Smart Placement・記録の設定・Hyperdrive）
open-next.config.ts / next.config.ts    変換の設定・セキュリティ見出し・CSP
scripts/hyperdrive.mjs  Deploy のたびに Hyperdrive を作る／直す
tests/             vitest（33 ファイル・420 件。PGlite で動く）
docs/              SPEC.md（仕様書）・SPEC.pdf・ARCHITECTURE.md（この文書）・PRELAUNCH.md
```

`src/lib` の主な部品：

ファイル	役割
`db.ts` / `schema.ts`	データベースへの入口（PostgreSQL か PGlite）と、表の形・版の上げ方（`SCHEMA_SQL`・`MIGRATIONS`）
`surveys.ts`	診断の作成・回答の受付・N 人目での結果の確定・自己回答・人数をふやす・期限・公開リンク・削除
`scoring.ts`	判定の計算（§6）と、回答から作る分析の材料（§7.5〜7.7）。計算の版 `ALGORITHM_VERSION`
`describe.ts`	保存した結果の数字を、表示する文に組み立てる（文章ではなく「規則と数字」を保存する方式）
`compare.ts`	2 人の結果を並べる・前回との比較
`deepAnalysis.ts` / `premium.ts` / `stripe.ts` / `premiumTerms.ts` / `supportStats.ts` / `fortune.ts`	特典つきの応援（くわしい分析・占い・支払い）
`push.ts` / `pushClient.ts`	Web Push の送信（サーバー）と登録（ブラウザ）
`groups.ts`	グループ診断
`guard.ts` / `http.ts`	入口の決まり（https 化・よそのサイトからの書き込みの拒否・JSON 以外の拒否）と API の共通部品
`admin.ts` / `passkeys.ts`	運営者の認証（合言葉・パスキー・セッション）
`inquiries.ts` / `visits.ts` / `funnel.ts` / `track.ts` / `itemStats.ts`	問い合わせ・リンクの開かれた回数・計測・問ごとの質
`image.tsx` / `site.ts` / `text.ts` / `client.ts`	シェア画像・公開アドレス・文字の扱い（呼び名の確かめ）・ブラウザ側の道具

4. データ

4.1 表

`src/lib/schema.ts` が正。表は最初の接続のときに自動で作られ、版（`schema_meta`）が古いときだけ、助言ロックを取って 1 つのトランザクションで更新する（起動のたびに `alter table` を流すと表全体に鍵がかかり、応答が 1〜4 秒遅れたため）。

表	中身
`surveys`	診断。`hostKey` は SHA-256 のハッシュ値だけ。状態・人数・期限・確定した結果（`result` jsonb）・公開リンク

表	中身
`responses`	回答。32 個の値（1〜7、0=わからない、−1=出していない問）・関係・端末ごとの乱数。日時は日にちまで
`push_keys` / `push_subscriptions`	通知の送信鍵（VAPID）と送り先（送ったら消す）
`survey_groups` / `group_members`	グループ診断
`inquiries`	問い合わせ（180 日で消す）
`payments` / `support_records`	特典つきの応援の支払いと記録
`admin_passkeys` / `admin_challenges` / `admin_sessions`	運営者のパスキー・1 回だけの乱数・セッション（ハッシュ値）
`visit_counts` / `funnel_counts` / `survey_stats` / `stats_responses`	計測（件数と日時だけ。呼び名・回答・鍵は入れない。400 日で消す）
`rate_limits`	同じ回線からの回数の制限
`schema_meta`	表の形の版・定時処理が最後に済んだ日時

4.2 保存する結果

結果は文章ではなく、**どの規則にどの数字を入れるか**（`algorithmVersion`・`n`・`type`・4 軸の極と割合・`insights` の規則と数字・自己回答の比較・関係ごとの型）を jsonb で保存し、表示のときにデータファイルの文と組み合わせる。文言を直しても結果の数字は変わらず、1 人分の点に使える細かい値（軸の合計点）は返さない。散らばり・「友達の多くがうなずいた行動」・くわしい分析は保存せず、見るたびに回答から作る（§11.5）。

4.3 端末に置くもの

ブラウザの保存領域に、作った診断の `hostKey` の一覧・通知の設定・前回の結果のまとめ・答えかけの回答を置く。二重回答を止める回答者の乱数は、その診断の API のパスにだけ送られる httpOnly の cookie（`tkr_<ID>`）。保存領域が使えない端末でも画面が壊れないようにし、管理用リンクを自分あてに保存するよう案内する（§8.3）。

5. 主な処理の流れ

5.1 診断を作る

`POST /api/surveys` が、`surveyId`・`hostKey`・（公開したときの）`shareId` を**それぞれ別の**暗号的に安全な乱数で作る。`hostKey` はこのときだけ平文で返し、データベースにはハッシュ値を置く。画面は `/h/share#{hostKey}` に移り、鍵は `#` より後ろにあるのでサーバー・ログ・Referer に残らない。

5.2 回答を受け付け、N 人目で確定する

`POST /api/surveys/{surveyId}/responses` は、1 つのトランザクションで次を行う。

1. `select … from surveys where id = $1 for update` で、その診断の行をほかの処理から止める

2. 状態が `open` で `response_count < target_count` かを確かめる（だめなら「締め切り」）
3. 深掘りの問の出し方（§3.5）と同じ決まりで答えの形を確かめ、`responses` に追加して `response_count` を 1 増やす
4. `target_count` に達したら、その場で §6・§7 の結果を計算して `result` に入れ、`status = 'completed'`・`completed_at`・`delete_at` を入れる
5. 確定する。通知は返事を返したあとに送る（`waitUntil`）

同時に何人送っても N 人を超えず、結果は N 人分のデータだけで 1 回計算される。受付中の診断について、結果・途中の集計・個々の回答を返す API は作らない。

5.3 結果を見る

`GET /api/host`（見出し `X-Host-Key`）が、保存した結果と、見るたびに回答から作る材料（散らばり・うなずいた行動・くわしい分析）を `HostView` として返す。画面はそれを `describe.ts` の文の型に差し込む。回答状況の画面は、見えているあいだだけ 10 秒ごとに取り直す。

5.4 通知

ホスト・答えた人が「通知を受け取る」を押したときだけ送り先を登録する。N 人目の回答で確定したら、Web Crypto だけで組んだ Web Push（aes128gcm・VAPID）で 1 回だけ送り、送り先を消す。送り先のアドレスは主要なブラウザの中継の会社のものだけを受け付ける。iPhone はホーム画面に追加したアプリの中だけで受け取れる（§11.10）。

5.5 定時処理

`wrangler.jsonc` の Cron（日本時間 3:17）が `worker.ts` を通して `/api/cron/cleanup` を `CRON_SECRET` 付きで呼ぶ。期限切れ（30 日、1 回だけ 30 日延長）・削除（確定から 180 日、期限切れから 30 日）・計測データの掃除（400 日）を 1 つずつ行い、済んだ日時を `schema_meta` に書く。`/api/health` は、これが 26 時間より古ければ 503 を返すので、見回りで止まったことに気づける。

5.6 公開カードとシェア画像

ホストが「公開リンクを作る」を押したときだけ `shareId` を作り、`/r/{shareId}` でタイプのカードと 4 軸だけを見せる（`noindex`）。画像は `next/og` で作り、縦（1080×1920）は結果の画面で長押し保存、横（1200×630）は OG 画像にだけ使う。画像には呼び名・タイプ・4 軸の割合・人数だけを入れ、`no-store` で端末に残さない。人数をふやすと公開リンクは取り消す。

6. API

方法	パス	使う人	中身
POST	`/api/surveys`	ホスト	診断を作る

方法	パス	使う人	中身
GET	`/api/surveys/{id}/public`	回答者	呼び名・状態・あと何人か（結果は返さない）
POST	`/api/surveys/{id}/responses`	回答者	回答を送る（§5.2）
POST/DELETE	`/api/surveys/{id}/push`	回答者	「結果が出ました」の通知の登録・解除
GET/DELETE	`/api/host`	ホスト	状態・結果／診断を消す
GET	`/api/host/summary`	ホスト	「つくった診断」の札だけ
PUT	`/api/host/self`	ホスト	自己回答（結果が出たあとも可。比較だけ作り直す）
POST	`/api/host/extend` / `target` / `share`	ホスト	期限を延ばす／人数をふやす／公開リンクを作る・取り消す
POST/DELETE	`/api/host/push` ・ `/api/host/push/status`	ホスト	「そろった」の通知の登録・解除・確認
GET	`/api/host/image?size=story|wide`	ホスト	結果画像（確定後）
POST	`/api/host/premium` / `gift` / `support` / `event`	ホスト	特典つきの応援・贈り物・応援の記録・結果の表示の計測
GET	`/api/og/{shareId}`	だれでも	公開した結果の画像
GET	`/api/push/key`	だれでも	通知の購読に使う公開鍵
POST	`/api/event` / `/api/visit` / `/api/contact`	だれでも	画面の操作の計測／リンクの開かれた回数／問い合わせ
POST	`/api/fortune`	合言葉を持つ人	タコ占い
POST	`/api/stripe/webhook`	Stripe	支払いの確定・返金
GET	`/api/cron/cleanup`	定時処理	`Authorization: Bearer {CRON_SECRET}`
GET	`/api/health`	見回り	`{ ok }` だけ
各種	`/api/admin/…`	運営者	問い合わせ・報告された診断の削除・計測・問の質・パスキー・ログイン
GET/POST	`/api/groups/…`	グループ	既存のグループの参照・参加・開始（新規作成は 410）

`/api/host` で始まる API は `X-Host-Key` 見出しで鍵を受け取る。書き込みの API は、よそのサイトからの呼び出し（`Sec-Fetch-Site` が `same-site`・`cross-site`、または `Origin` が合わない）と、`application/json` 以外の本文を入口で断る。同じ回線からの回数は `rate_limits` で制限する。

7. 安全と匿名性の守り

- **鍵**: `surveyId`・`hostKey`・`shareId` は別々の乱数（128 ビット以上）。`hostKey` はハッシュ値だけを保存し、URL の `#` より後ろと見出しだけで運ぶ。すべてのページで `Referrer-Policy: no-referrer`。Workers Logs の通信ごとの記録（見出しが丸ごと残る）は止める（`observability.logs.invocation_logs: false`）
- **匿名性**: 人数の下限 3 人、関係ごとの表示は 3 人以上、問ごとの分析は 3 人以上に出した問だけ、軸の合計点は返さない、回答の時刻は日にちまでで画面に出さない、関係の並びは名前順（§8.5）

- **入口の決まり**（`worker.ts`・`guard.ts`）：`http://` は `https://` へ、`Strict-Transport-Security`、よそのサイトからの書き込みの拒否、JSON 以外の本文の拒否
- **見出し**（`next.config.ts`）：`Content-Security-Policy`（`default-src 'self'`。Next.js が書く小さなスクリプトのため `'unsafe-inline'` は外せない）、`X-Frame-Options: DENY`、`X-Content-Type-Options: nosniff`、`Permissions-Policy`
- **入力**：呼び名は 1〜12 文字（見た目の文字数と文字コードの数の両方に上限）、禁止語（表記ゆれ・まぎらわしい文字・見えない文字を正規化して確かめる）、HTML として解釈しない
- **運営者**：合言葉（`ADMIN_TOKEN`）はかかる時間から分からない比べ方、10 回まちがえたら 1 時間止める。パスキー（公開鍵だけ保存、本人確認を毎回）、12 時間の cookie セッション（`httpOnly`・`SameSite=Strict`・`/api/admin` だけ）。パスキーの登録・取り外しは毎回合言葉を求める
- **検索**：公開カードと 2 人を並べるページは `noindex`

8. 公開と運用

8.1 流れ

```
push（main） ──→ CI（lint・format・typecheck・test×2（PGlite・PostgreSQL 16）・build・OpenNext build）
                  └── 成功したときだけ Deploy（workflow_run）
                      1. scripts/hyperdrive.mjs：Hyperdrive「takoshin-db」を作る／接続先を直す（キャッシュなし）
                      2. opennextjs-cloudflare build → populateCache remote（前もって作れるページを静的ファイルへ）
                      3. wrangler deploy --secrets-file（DATABASE_URL・CRON_SECRET・ADMIN_TOKEN・Stripe の鍵を同じ 1 回で）
Healthcheck（3 時間ごと）：トップ・作る画面・一覧・/api/health を各 3 回まで試す
```

- **設定**：GitHub のシークレット `CLOUDFLARE_API_TOKEN`・`CLOUDFLARE_ACCOUNT_ID`・`DATABASE_URL`・`CRON_SECRET`・`ADMIN_TOKEN`、任意で `STRIPE_SECRET_KEY`・`STRIPE_WEBHOOK_SECRET`。変数 `NEXT_PUBLIC_SITE_URL`、任意で `NEXT_PUBLIC_SUPPORT_URL`・`SUPPORT_PRICE_YEN`・`SUPPORT_TIERS_YEN`
- **アドレス**：独自ドメイン `takoshin.daigakunote.com`（`wrangler.jsonc` の `routes`）。`workers.dev` も止めない（送り済みの招待リンクのため）。転送はしない（端末の保存領域がアドレスごとに分かれるため）。人に送るリンクだけ `NEXT_PUBLIC_SITE_URL` で作る
- **性能**：Worker は Smart Placement でデータベースの近くに置く。Hyperdrive で接続を速くし、結果のキャッシュは切る。前もって作れるページは静的ファイルから返す（毎回作ると最初の応答に 0.3〜1.8 秒かかっていた）。同じ表示の中で同じ問い合わせを 2 回しない（React の `cache`）
- **記録**：Workers Logs は `console.log`／`console.error` だけ（定時処理の結果など）。個人や回答の中身は記録に入れない
- **見回り**：`/api/health` はデータベースと定時処理の両方を確かめる。GitHub の定時の起動は遅れたり飛んだりするので、すぐ気づくには外部の監視サービス（UptimeRobot など）か Cloudflare のヘルスチェックを足す（手順は README）
- **バックアップ**：Neon の復元できる期間は契約のプランで決まる。管理画面で確かめる

8.2 手元での開発

```
npm install
npm run dev          # http://localhost:3000。DATABASE_URL がなければ .data/pglite に PGlite を作る
npm test             # vitest（PGlite）。DATABASE_URL=postgres://… npm test で本物の PostgreSQL
npm run lint / typecheck / format:check / build
npm run cf:preview   # Workers と同じ環境で試す（.dev.vars に DATABASE_URL など）
```

9. 変える・増やすときの手順

変えるもの	手順
表や列を足す	`schema.ts` の `SCHEMA_SQL` に `if not exists` で書き足し、`MIGRATIONS` に次の版を足す。既存データの直しも同じトランザクションで。仕様書 §11.2 に版の記録を書く
問の文を直す	軸・向き・聞く行動を変えなければ `question_version` は上げない（受付中の診断を期限切れにしないため）。意味が変わるなら版を上げ、古い版の診断は定時処理で期限切れにする
判定の計算を変える	`ALGORITHM_VERSION` を上げる。保存ずみの結果は前の版のまま表示する（`describe.ts` は版ごとの違いを吸収する）
16 タイプの本文を直す	`types.json` を直し、仕様書 §12 を同じ内容にする（§12 は `types.json` と同じ内容という決まり）
画面の文言・ふるまいを変える	仕様書の該当する節と、冒頭の版の記録を更新する。`docs/SPEC.pdf` を作り直す（README の手順）
依存部品を上げる	公開から 14 日たった版だけ。OpenNext と wrangler はまとめて上げる
版を上げる	機能を変えたら `npm version <版> --no-git-tag-version`。main に入ると CI → Deploy が自動で公開する

10. 今回は作らないもの・将来の案

グループ診断の新規作成は 1.51 で終了し、`POST /api/groups` は 410 を返す（既存のグループは期限まで使える）。将来の案（友達どうしでいつでも答えられる仕組み・友達のタイプ帳）は仕様書 §17 にある。